AI-ASSISTED RESEARCH METHODOLOGY

AI-Assisted Research Programming

From Vibe Coding to Spec-Driven Development

A Practical Guide for Researchers

PRESENTER

[Jiale Guo]

2024_Fall_MSc_Geoinformatics Engineering
Politecnico di Milano

DATE

2025-12-28

1.0 Programming Evolution

From Humans Adapting to Machines to Machines Serving People

Evolutionary Law

Programming abstraction layers rise continuously; HCI approaches natural language.

NEXT STEP

Prompt Engineering Is The New Coding

1.1 Programming Paradigm Revolution

From 'Writing Code' to 'Describing Requirements'

Core Shift

Traditional = Human → Machine Language

AI Programming = Human → AI → Collaborative Code

From "Syntax Recall"

To "Prompt Engineering"

Key Dimension Comparison

Dimension	Traditional Programming	AI Programming	Nature of Change
Core Skills	Syntax recall, API lookup	**Requirement description, Logic validation**	Memory → Expression
Learning Curve	Steep Months-Years	Gentle Days-Weeks	Lower Barrier
Dev Speed	Slow (Manual write + Manual debug)	**Fast (Instant generation + Iterate)**	Efficiency Leap
Scope	All scenarios (incl. low-level drivers)	80% Routine research tasks	Pareto Principle

AI Strengths

- Standard algorithms (Sorting, Stats, GIS ops)
- Code translation (Python ↔ R)
- Error diagnosis & Fix suggestions (Explain & Fix)
- Automated documentation & comments

AI Limitations

- Complex domain logic (Needs deep knowledge)
- Novel algorithm design (Non-standard/New)
- System-level architecture decisions
- Extreme performance optimization (Hardware level)

1.2 Vibe Coding

Definition, Strengths & Pitfalls

Vibe Coding

Natural language conversation-driven, exploratory, intuition-based programming method characterized by rapid trial-and-error iteration.

Three Core Features

Low Barrier
No complex syntax required, natural language is code

High Speed
Idea to code in minutes, rapid validation

Strong Exploration
Discover unknown possibilities via dialogue

Recommended

- ✓ Code < 200 lines
- ✓ One-off scripts
- ✓ Rapid prototypes
- ✓ Learning new stacks

Avoid

- × Code > 500 lines
- × Long-term maintenance
- × Team projects
- × Core algorithms

Three Fatal Flaws

Scale Disaster
500+ lines become a "patch pile", lacking architecture, maintenance cost rises exponentially.

Understanding Drift
Term ambiguity (e.g., "building density" definitions), AI lacks context and won't clarify proactively.

Doc Drift
Code changes without doc updates lead to inconsistency, rendering documentation useless.

1.3 Solution Preview

Keep Vibe Speed, Add Spec Reliability

Core Shift: From Dialogue-Driven to Spec-Driven (Contract)

1

Vibe Coding

DIALOGUE-DRIVEN

High Speed

Intuitive

Unstable Quality

→

2

Spec-First

PRIORITY ON SPECS

Def-First

Anchored Iterations

Reliable Quality

→

3

Spec-as-Source

SPECS AS CODE

Auto-Gen Code

100% Consistent

Collab Foundation

Vibe Coding = Verbal House Building

"Ad-hoc changes, building as you speak"

Result: Crooked house, missed details

Spec-Driven = Blueprints First

"Blueprints define all details and standards"

Result: Quality assured, reproducible

> *"Vibe Coding is Fast Food—tasty but unhealthy. Spec-Driven is Healthy Fast Food—fast AND reliable."*

2.1 Tool Panorama (2025): Four Categories

2025 Mainstream AI Programming Tools Landscape

AI-Native IDEs

Complete development environment, deeply integrated AI

Cursor | Windsurf | Antigravity | Trae | Kiro | Augment

- For medium-large projects
- Deep context understanding
- Agent mode: Autonomous development

CLI Tools

Geek's choice, scripts & automation

Claude Code | Codex CLI | Gemini CLI | Droid

- For complex reasoning & deep logic
- Automation & task decomposition
- Huge context window (200K - 1M)

Cloud Platforms

Online development, zero configuration

Replit | bolt.new | Lovable | Orchids

- Zero setup, browser-ready
- Real-time full-stack preview
- Ideal for teaching & prototypes

Auxiliary Tools

Specific task enhancement & visualization

Augment Code | Zread | NotebookLM | Mermaid Chart

- Context engine enhancement
- Document interpretation
- Architecture visualization

Core Advice: Combinations beat single tools

Recommended Research Combo: AI-Native IDE (for building projects) + CLI Tools (for deep reasoning)

Updated 2025 Data

2.2 AI-Native IDEs (Part I)

Cursor / Windsurf / Antigravity Comparison

Cursor

BENCHMARK

★★★★★

$20 Pro / $40 Business

Claude 4.5 Sonnet / GPT-5.2

Agent Mode: Autonomous project-level dev

Composer: Multi-file editing & refactoring

@Symbols: Precise context referencing

.cursorrules: Project-specific norms

Research Value: Complex Algorithms

Windsurf

VALUE KING

★★★★☆

$15/mo (~1000 credits)

Cascade Flow / Claude4.5 / GPT 5.2

Cascade Flow: Iterative coding system

Context: Strong multi-file awareness

Cost: 25% cheaper than Cursor

Agentic: First agent-IDE (2024 debut)

Research Value: Data Scripts & Prototyping

Antigravity

FREE GEM

★★★★★

Completely FREE

Claude Opus 4.5 + Gemini 3.0 Pro

Multi-Agent: Async task processing

Cross-Platform: Sync VSCode/Web/Terminal

Ecosystem: Seamless Google integration

Zero Cost: Best for students/academics

Research Value: Students & Exploration

2.2 AI-Native IDEs (Part II)

Trae / Kiro / Augment: Specialized Solutions for 2025

Trae

FREE (China)

Doubao / Claude 4.5

- ByteDance Localized Solution
- 10 fast + 50 slow requests/mo
- No proxy required, direct access
- Real-time Web preview
- Best for Chinese docs & teaching

LOCALIZATION

★★★★☆

Kiro

$20 / $40 / $200

Model Agnostic

- Spec-Driven Development (SDD)
- Auto-maintained Memory Bank
- Production-grade quality assurance
- Enforced "Docs First" workflow
- Ideal for large collaboration

SPEC-DRIVEN

★★★★★

Augment

$100 / mo

Context Engine + GPT-5.2

- Extreme Context Engine (RAG+)
- 208,000 Credits monthly
- Native MCP tool integration
- SOC 2 Type II Enterprise Security
- Best for huge legacy codebases

CONTEXT KING

★★★★★

2.2 CLI Tools Deep Dive

Claude Code / Codex CLI / Gemini CLI / Droid

Claude Code

PRO/MAX

Core Model	Claude 4.5 Sonnet / Opus
Pricing	$20 Pro / $200 Max

- ✓ Project Memory
- ✓ 200K Context
- ✓ Multi-step Decomp.
- ✓ Deep Code Analysis

Best For: Large project refactoring, precise paper algorithm implementation

Codex CLI

PLUS

Core Model	GPT-5.2
Pricing	$20 Plus / $200 Pro

- ✓ Native Multimodal
- ✓ Real-time Web Search
- ✓ Plugin Ecosystem
- ✓ Fast API Integration

Best For: Rapid prototyping, API integration, multimodal tasks

Gemini CLI

FREE

Core Model	Gemini 3.0 Pro
Pricing	Free (60 req/min)

- ✓ 1M Context Window
- ✓ Gemini 3.0 Flash
- ✓ Zero Cost Processing
- ✓ Search Integration

Best For: Zero budget projects, large text analysis, students

Droid

FACTORY

Vendor	Factory AI
Pricing	Subscription / API Key

- ✓ Deep CI/CD Integration
- ✓ Code Isolation
- ✓ Automated Pipelines
- ✓ Enterprise Compliance

Best For: Enterprise DevOps automation, batch refactoring

CLI Core Value: Pipeline workflows & automation — "Refactor this" for huge codebases without manual intervention.

```
gemini "refactor this" < main.py > main_v2.py
```

2.4 Cloud Platforms: Replit / bolt.new / Lovable / Orchids

Zero-setup online environments for rapid prototyping & teaching

Replit

Free / Paid

- Browser-based IDE with zero environment setup requirement
- Real-time collaborative editing (Google Docs style)
- Instant Node.js runtime & one-click deployment

Best for: Teaching, Demos, Quick Prototyping, Sharing Scripts

bolt.new

Free / Paid

- Modern full-stack environment based on WebContainers
- Instant run & live preview with hot-reloading
- Seamless StackBlitz integration for web apps

Best for: Web App Development, Frontend Demos, Full-stack Projects

Lovable

~$20/mo

- Automated full-stack app generation (Product-focused)
- High-quality UI design generation out-of-the-box
- Integrated database & backend logic setup

Best for: Rapid MVPs, Non-technical Founders, Tool Building

Orchids

Free Trial

- Leader in UI Bench benchmarks for interface generation
- Specialized in building complex interactive interfaces
- Ideal for creating visualized research tool interfaces

Best for: Research Dashboards, Experiment Control Panels, Complex UIs

Key Insight: Accessibility vs Performance

Excellent for cross-device work and teaching. Note: Be mindful of upload/download bottlenecks when handling large local datasets (e.g., GB-sized imagery).

Cloud-First

2.3 Tool Selection Decision Tree (2025)

Strategies for Tool Selection: Budget × Scenario × Skill

By Budget

BUDGET

Zero Budget

Antigravity + Trae + Gemini CL

Free powerful tools from Google & ByteDance covering full workflow

Low Budget <$20

Windsurf ($15)

High value with Cascade Flow, ideal for students

Standard $20-40

Cursor Pro ($20)

Mature IDE + Free powerful CLI, industry standard

High End $100+

Augment / Kiro

Extreme context & Spec-Driven Dev, enterprise choice

By Scenario

SCENARIO

Scripts <500 Lines

Cursor + Jupyter

Best for interactive analysis & rapid hypothesis validation

Mid Project 500-2000

Windsurf / Cursor

Balance context understanding & cost, fast iteration

Large System >2000 Lines

Augment / Kiro

Huge context engine or Spec-Driven for robust architecture

Algorithm Research

Claude Code + Antigravity

Deep reasoning & multi-model collab for complex algorithms

By Skill

SKILL LEVEL

Beginner

Trae / Replit

User friendly / Zero setup, out-of-the-box experience

Python

Cursor / Windsurf

Maximize efficiency with existing knowledge, seamless migration

CLI-First

Claude Code / Gemini

Pure keyboard speed, hacker-style development experience

Vibe Coding Meta-Methodology

Philosophy • Methodology • Tactics • Tools

Philosophy (Dao)

Core Mindset & First Principles

- AI First: If AI can do it, humans shouldn't.
- Context is King: Garbage in, garbage out.
- Structure First: Plan before implementation.
- Occam's Razor: Do not add code unless necessary.

Methodology (Fa)

Systematic Implementation Principles

- Explicit Boundaries: One-sentence goal & non-goals.
- Orthogonality: Non-repetitive functional design.
- Docs as Context: Copy over writing; interface first.
- Interfaces First: Decompose modules by responsibility.

Meta-Methodology

Tactics (Shu)

Practical Execution Skills

- Constraints: State clearly what can/can't change.
- Debug Strategy: Expected vs Actual + minimal repro.
- Testing: AI generates tests, humans assert validity.
- Reset Session: Start fresh when code grows too large.

Tools (Qi)

Tool Selection & Combination

- IDEs: Cursor / Windsurf / Kiro
- CLI: Claude Code / Gemini CLI
- Models: Claude 3.5 / GPT-4o / Gemini 2.5

→ *See "Tool Panorama" for detailed comparison*

3.1 Prompt Engineering: Art of Precision Communication

5W1H Framework & Scientific Research Template

Comparative Experiment: Why AI "Doesn't Understand"?

Vague Description (Vibe Coding) — Result: Confusion

User: "Analyze this data for me"

AI Thinking: - Which dimension? (Stats? Regression?) - Which tool? (Pandas? Numpy?) - Output format? (Print? CSV? Plot?)

Precise Description (Prompt Engineering) — Result: Success

User: "Read data.csv, calculate **Mean/Median/Std** of 'pop' column. Use Matplotlib to plot a bar chart. Save as output.png with **300dpi**."

AI Understanding: Input: data.csv | Task: Stats + Plot Stack: Matplotlib | Output: 300dpi PNG

5W1H Structural Framework

WHAT

What is the specific task?Core function definition

WHY

What is the business context?Context for intent

WHO

Who is the target audience?Style & Depth level

WHEN

Batch processing or real-time?Performance & Arch

WHERE

Data source and output location?I/O Path definition

HOW

Technical constraints & quality?Libs/Specs/Precision

Scientific Research Template

Task Description: [Verb] + [Object] + [Purpose]

Data Context: Source / Scale / Format

Processing Steps: Step 1... Step 2... (Logic Chain)

Output Requirements: Format / Precision / Metrics

Technical Constraints: Required Libs / Coding Style

Test Verification: Provide Mini Test Case

Specification-Driven Development (SDD) Overview

Core Principles, Basic Workflow & Three-Level Practice Path

Core Principle

Spec is the project's "Source of Truth", Code is merely the "Implementation Artifact".

Change Request → Update Spec First → AI Updates Code

Level 1

Spec-First Specification First

Ideal starting point for one-off tasks. Clearly define requirements first, then let AI generate code. Archive specs after generation.

- ✓ Applicable: <200 line scripts
- ✓ Higher quality than Vibe Coding

Level 2

Spec-Anchored Specification Anchored

Engineering standard. Specs and code are updated synchronously, serving as a long-term maintenance anchor. Requires Git management.

- ✓ Applicable: 500-2000 line projects
- ✓ Essential for team collaboration

Level 3

Spec-as-Source Specification as Source

Extreme automation. Edit only specs; code is fully auto-generated and strictly read-only, ensuring absolute consistency.

- ✓ Applicable: Structured/repetitive logic
- ✓ 100% consistency guarantee

CORE BENEFITS: Fewer Detours · Reproducibility · Seamless Collab · Maintainability

Level 1: Spec-First

Entry-level Practice: Fast, Efficient, Traceable

Standard Workflow

1. Define Requirements & Goals
2. Write Spec Document (Markdown)
3. AI Generates Code Based on Spec
4. Manual Review & Simple Testing
5. Task Complete, Archive Spec

APPLICABLE SCENARIOS

- Small Scripts (<200 lines)
- One-off Data Processing
- Quick Prototype Validation
- Learning New Tech Stacks

spec_poi_density.md

Example Spec

```
# Feature: POI Kernel Density Analysis

## 1. Requirements Overview
Calculate kernel density of urban POIs, generate heatmap...

## 2. Input Data
-pois.geojson     : GeoDataFrame, Point Geometry, EPSG:4326
-boundary.shp     : Study Area Boundary Polygon

## 3. Output Deliverables
-density.tif      : GeoTIFF Density Raster (Float32)
-heatmap.png      : Visual Heatmap (300dpi, with legend)

## 4. Algorithms & Parameters
- Method: Gaussian Kernel Density Estimation
- Bandwidth: Use Scott's rule adaptive calculation

## 5. Code Requirements
- Function:     calculate_kde(gdf, bandwidth=None) -> raster
- Must include Google-style docstrings
```

Pros

Higher quality than pure chat; Documented and traceable

Cons

Spec can become outdated; Not for complex iterations

Level 2: Spec-Anchored

Memory Bank Architecture & Synchronization Workflow

Spec as Long-Term Anchor

For medium projects (500-2000 lines), code exceeds mental context limits. The Specification serves as the external memory and "source of truth" that must be kept in sync with code.

Synchronization Workflow

New Requirement / Bug Found

↓

Update Spec Documentation First

↓

AI Updates Code Based on New Spec

Team Collaboration Consensus

Long-term Maintenance

MEMORY BANK ARCHITECTURE

Project Structure

```
project_root/
├── .ai-context/          # AI Long-term Memory
│   ├── constitution.md   # Non-negotiable rules
│   ├── architecture.md   # System design
│   └── glossary.md       # Domain terms
├── specs/                # Feature Specifications
│   └── feature-001/
│       ├── requirements.md
│       └── tests.md
└── src/                  # Implementation Code
```

Level 3: Spec-as-Source

Extreme Automation via "Read-Only" Code

Extreme Automation

Code as Build Artifact

Treat the Spec as the source code. Generated code is a "build artifact" that is never manually edited. This guarantees 100% consistency between documentation and implementation.

data_models.yaml — EDITABLE SOURCE

```
User:
  fields:
    – id: UUID, primary_key
    – email: String, unique, required
  methods:
    –validate_email()      : bool
    –send_notify(msg)      : void
```

AI COMPILER → AUTO GEN

models.py — DO NOT EDIT

```python
# AUTO–GENERATED – DO NOT EDIT
from dataclasses import dataclass
from uuid import UUID

@dataclass
class User :
    id: UUID
    email: str

    def validate_email (self) –> bool:
        """Validate format logic..."""
        pass
```

BEST USE CASES

- Data Models
- API Schemas
- Config Management
- Multi-Platform Sync

Memory Bank & Glossary

Eliminating ambiguity and ensuring long-term consistency

AI Long-Term Memory

Use structured documentation as an external "Memory Bank" to maintain context across sessions.

Add .ai-context/ to .gitignore but include in AI context

Project Structure

```
project/
  .ai-context/
    constitution.md      # Rules
    architecture.md      # Design
    glossary.md          # Terms
    coding-standards.md
  specs/
    feature-001/
      requirements.md
      tests.md
  src/                   # Code impl
```

constitution.md (Highest Priority)

 # Project Constitution: Urban POI System

Core Principles (Non-negotiable)

1. Data First: Use real data only, no fabrication.
2. Reproducibility: Every step must have a script.
3. Quality: Correctness > Speed.

Forbidden Items

- NO global variables
- NO 'from module import *'
- NO hardcoded paths

glossary.md (Eliminate Ambiguity)

 # Terminology Glossary

Building Density

Definition: Number of buildings per unit area.

Formula: density = count(buildings) / area(km^2)

Note: ≠ Building Coverage Ratio (Area Ratio)

Kernel Density Estimation (KDE)

Impl: scipy.stats.gaussian_kde

Param: bandwidth (default: Scott's rule)

Comparative Experiment: Two Paths for the Same Task

Vibe Coding vs Spec-First Performance Data

Task Goal | Calculate 15-min urban road network isochrones | Input: Road SHP + Origin Point | Output: GeoDataFrame (Polygon)

Method A: Vibe Coding

"Intuition-driven, ad-hoc changes"

1. Gen code v1 (AI used Euclidean dist)
2. Debug: Switch to network dist (No NetworkX)
3. Debug: Add NetworkX (Ignored speed limits)
4. Debug: Add speed limits (Extremely slow)
5. *After 7 iterations, code structure is messy...*

60 min TOTAL TIME	**7** x DEBUG ROUNDS
DOC COMPLETENESS	**60** pts CODE QUALITY

Barely usable, hard to maintain

Method B: Spec-First

"Contract-driven, spec before code"

1. **Write Spec Document (5 min)**
2. AI generates full code at once (2 min)
3. Run unit tests: Passed (1 min)
4. AI adds README docs (2 min)
5. Final validation, no major logic errors

10 min TOTAL TIME	**0** x DEBUG ROUNDS
DOC COMPLETENESS	**90** pts CODE QUALITY

High Quality, Reproducible

Key Insight: Spending 5 mins on Spec saves 6x development time.

EFFICIENCY BOOST: 600%

4.1 Context Management

Layered References & The "Minimal Sufficient Context" Principle

The Core Challenge

Context Overflow: In large projects (1000+ lines), providing all code to the AI exceeds window limits, causing "hallucinations" or "amnesia".

Wrong: Dumping the entire codebase.

The Golden Rule

Minimal Sufficient Context: Like a surgical operation, provide only the exact documentation, interfaces, and code snippets required for the current task.

Right: Precise, layered selection.

Layered Context Strategy

Project Constitution

Global rules, tech stack, forbidden patterns. Always Included (e.g., constitution.md).

Architecture & Modules

Module responsibilities, interfaces, data flow. On-Demand (e.g., architecture.md).

File Level

Specific source files relevant to the active task. Targeted (e.g., src/data_loader.py).

Function/Class Details

Granular snippets, specific functions to modify. Pinpoint (e.g., class DataProcessor).

Pro Tip: In tools like Cursor, combine references (e.g., @Constitution + @File) to build the perfect context.

Cursor @ Reference System

Practical Examples for Precise AI Context Management

Reference Commands

@file	Reference complete file contente.g., @data_loader.py
@folder	Reference folder structure (no content)e.g., @src/analysis
@code	Reference specific function/class (Precise)e.g., @calculate_kde
@docs	Reference indexed documentatione.g., @Pandas (No context switch)
@web	Browse specific URL for infoe.g., @https://stackoverflow...
@git	Reference commits or diffse.g., @diff (Review changes)

High-Efficiency Prompt Construction

Context Building (The "Memory")

```
@.ai-context/constitution.md

# Level 0: Project Rules
@specs/kde/requirements.md

# Level 1: Feature Specs
@code:calculate_density

# Level 3: Target Function
```

Execution Prompt (The "Action")

Based on the above specs, optimize `calculate_density`
: 1. Objectives: – Optimize for 100k points to **< 5s** – Replace loops with `scipy.spatial.cKDTree`
2. Constraints: – Maintain **100% Type Hints** – Follow **Google Style Docstring**

Best Practices

Avoid @Folder Abuse: Large folders consume context window rapidly.

The "Combo": Always include @constitution.md for consistency.

Precision First: Use @code over @file when modifying specific functions.

4.2 Structured Debug Workflow

Stop "Blind Fixes", Become a Diagnostic Expert

⚠ Traditional Debug: Throw error to AI → Random guesses → Fix breaks more things

1 Locate Problem

SYMPTOMS

Write Debug Report
Record Repro Steps
Env & Exclusions

2 min

2 Root Cause

ANALYSIS

List 3 Likely Causes
Request "Diagnostics"
Pinpoint Culprit

1 min

3 Fix Design

PLANNING

Generate Options A/B
Compare Pros & Cons
Choose Minimal Impact

1 min

4 Implement

VERIFICATION

Atomic Modification
Run Regression Tests
Update Docs (if needed)

1 min

Expert Prompt Template

```
@debug_report.md # Reference your symptom report
"Don't just ask AI what to do, ask it to analyze causes + design solutions."
    Analyze: List 3 most likely causes for this error, ranked by probability.
    Diagnose: Provide verification steps for each cause (e.g., print vars).
    Fix: After confirmation, design a minimal-change fix and explain why.
# NOTE: Do not rewrite whole files directly. Analyze first.
```

Expected Outcome

Get logic-based analysis instead of blind code patches.

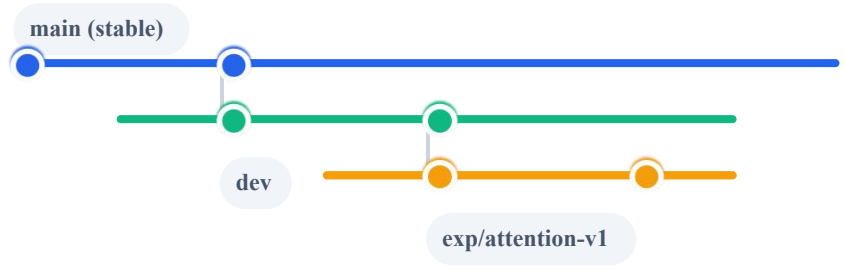
Stop "Vibe Debugging"

4.3 Version Control & Experiment Management

Research Workflow Integration

Git Workflow for Research

Structured branching strategy to isolate stable code from experimental features.



AI-Assisted Commit Message:

"I added attention module and fixed memory leak..."

```
feat(model): add hierarchical attention module

fix(data): resolve data loader memory leak
– Implement HierarchicalAttention class
– Add garbage collection after epoch
```

Experiment Tracking (W&B)

Move beyond Excel sheets. Use Weights & Biases for reproducible, visual tracking.

Hyperparameters
lr, batch_size, optimizer, architecture

Metrics & Performance
Train/Val Loss, Accuracy, F1 Score curves

Artifacts
Best Model (.pt), Log files, System specs

AI Prompt: "Add W&B logging to training loop"

```python
import wandb

wandb.init(project="urban-flow-pred", config={
"learning_rate": 0.001,
"architecture": "ResNet-50"
})

# ... inside training loop ...
wandb.log({"loss": loss, "accuracy": acc})
```

4.4 Key Practical Tips

6 Battle-Tested Tactics for Better AI Code Quality

1 Step-by-Step Description

Break complex tasks into logical chains. Avoid overwhelming the AI with too much at once.

BAD VS GOOD

"Data Analysis"

"Step 1: Read CSV; Step 2: Stats; Step 3: Plot"

2 Provide Examples (Few-shot)

Provide specific input/output samples for the AI to mimic exact formats.

PROMPT EXAMPLE

Input: {'name': 'BJ', 'pop': 2154}

Output: 'Beijing Population: 21.54M'

3 Role Setting

Assign expert personas to activate domain-specific knowledge weights.

PROMPT EXAMPLE

"You are a GIS expert, proficient in GeoPandas spatial indexing..."

4 Explicit Constraints

Set boundaries to prevent hallucinations or unwanted libraries.

PROMPT EXAMPLE

"Use matplotlib only, no seaborn; processing time < 10s"

5 Iterative Optimization

Do not aim for perfection in one shot. Refine in rounds.

WORKFLOW

R1: Basic Function → R2: Error Handling → R3: Perf → R4: Docs

6 AI Self-Review

Use AI's reflection capability to spot bugs before running code.

PROMPT EXAMPLE

"Please review the code above for potential memory leaks & edge cases."

Follow these 6 tips to reduce 80% of invalid dialogue & debugging time.

Desk Checklist

Summary: Cognition × Methods × Tools

AI-Assisted Research Programming Recap

Three Core Cognitions

Paradigm Shift: From "Writing Code" to "Describing Needs"

Mode Choice: Vibe Coding for speed, Spec-Driven for reliability

Tool Mindset: No "best tool," only best combinations

Three Practical Methods

Prompt Engineering: 5W1H framework & role constraints

Dev Levels: Spec-First / Anchored / As-Source three tiers

Context: Layered management & precise refs (@docs, @code)

Three Tool Strategies

Zero Budget: Antigravity + Trae (Free & Powerful)

Standard: Cursor Pro ($20/mo) + Gemini CLI

Enterprise: Kiro (Spec-Driven) + Cursor Team

> ***"AI is the copilot; you are the pilot. Master specs & tools for 10x productivity."***
>
> Always remember: Validate, Test, and Think Critically.